

VisionGuard: Driver Drowsiness & Distraction Detection

VISIONGUARD: AI-BASED DRIVER DROWSINESS AND DISTRACTION DETECTION

Academic Project Report

Submitted by: Yash Goyal

Reg Number: 24BAI10100

Course: B.Tech | **Subject:** Computer Vision (Flipped Course Evaluation)

Session: 2025 - 2026 | **Date:** 15/09/2026

VITyarthi - Build Your Own Project

2. INTRODUCTION

2.1 Project Overview

VisionGuard is a real-time Computer Vision application designed to monitor driver alertness, detect early signs of physiological fatigue, and identify visual inattention. By analyzing live video feeds from an in-cabin webcam or camera, VisionGuard computes dense facial geometry, evaluates scale-invariant eye and mouth aspect ratios, estimates 3D head orientation via Perspective-n-Point (SolvePnP), and issues multi-tier visual and auditory safety alerts.

2.2 Purpose

This project was developed as part of the VITyarthi flipped course evaluation to demonstrate practical application of core Computer Vision concepts, geometric feature extraction, spatial 3D pose estimation, temporal state machine modeling, and software engineering architecture.

2.3 Scope

The application focuses on core real-time driver monitoring functionality including facial landmark detection (MediaPipe 468 3D vertices), scale-invariant Eye Aspect Ratio (EAR) computation, Mouth Aspect Ratio (MAR) yawn analysis, SolvePnP 3D head pose estimation, multi-cue risk scoring, threaded auditory alert dispatch, and session telemetry persistence (CSV/JSON).

3. PROBLEM STATEMENT

3.1 Current Challenges

Transportation sectors and motorists face critical safety challenges related to driver impairment:

- Manual observation or passenger monitoring of driver fatigue is impossible during solitary driving.
- High fatality rates caused by involuntary micro-sleep episodes lasting 1 to 3 seconds.
- Visual distraction (looking at smartphones, navigation displays, or side windows) causing delayed hazard reaction times.
- Existing vehicular telemetry systems (lane departure, steering torque) operate reactively after lane drift has already occurred.
- Wearable sensors (EEG headbands, pulse oximeters) are intrusive and uncomfortable for drivers.
- High false alarm rates in early prototypes caused by speech and normal blinking.

3.2 Impact

These challenges lead to:

- Over 1.35 million annual road traffic deaths worldwide according to the World Health Organization (WHO).
- Severe economic and freight losses for commercial logistics and transport operators.
- Increased highway collision risks during nighttime long-haul journeys.
- Difficulty in identifying long-term driver fatigue patterns.
- Poor driver shift data organization and absence of objective safety auditing telemetry.

3.3 Proposed Solution

A non-intrusive, proactive Computer Vision Driver Monitoring System (DMS) that tracks facial biometrics in real time, applies temporal filtering to reject false positives, calculates composite risk, and issues immediate auditory and visual warnings prior to collision risk.

4. FUNCTIONAL REQUIREMENTS

4.1 Face and Facial Landmark Perception Module

- **FR-1: Frame Acquisition:** The system shall acquire video frames from hardware webcams or video files at a target rate of 30 FPS.
- **FR-2: Face Detection:** The system shall detect human faces in video frames using an anchor-based detector (BlazeFace).
- **FR-3: Landmark Localization:** The system shall extract 468 dense 3D facial landmarks and map normalized coordinates to image pixel coordinates.
- **FR-4: Facial Loss Handling:** The system shall transition cleanly to a `NO FACE` state when landmarks are temporarily lost without crashing.

4.2 Ocular & Drowsiness Tracking Module

- **FR-5: Eye Aspect Ratio (EAR):** The system shall calculate EAR across left and right eyes based on the 6-point Soukupová & Mach formulation.
- **FR-6: Blink Counting:** The system shall count intentional blinks that close and reopen within 1 to 4 frames.
- **FR-7: Prolonged Closure Detection:** The system shall trigger a drowsiness condition if eye closure persists for 15 continuous frames (approx 0.5s).

4.3 Yawn Detection Module

- **FR-8: Mouth Aspect Ratio (MAR):** The system shall calculate MAR measuring vertical oral aperture relative to horizontal mouth width.

- **FR-9: Speech False-Alarm Rejection:** The system shall require the mouth to remain open above threshold for ≥ 15 continuous frames before confirming a yawn.
- **FR-10: Yawn Refractory Cooldown:** The system shall enforce a 60-frame cooldown period to prevent multiple counts for a single yawn episode.

4.4 3D Head Pose & Distraction Module

- **FR-11: SolvePnP Pose Estimation:** The system shall solve the Perspective-n-Point problem using 6 canonical 3D facial coordinates to estimate head rotation and translation.
- **FR-12: Euler Angles Extraction:** The system shall decompose rotation vectors into Pitch, Yaw, and Roll angles.
- **FR-13: Distraction Detection:** The system shall flag driver distraction when head rotation exceeds $|\theta_{\text{yaw}}| > 25^\circ$ for >20 continuous frames.
- **FR-14: 3D Gaze Ray Projection:** The system shall project a 3D directional vector from the nose tip to visualize gaze direction.

4.5 Multi-Cue Risk Engine Module

- **FR-15: Linear Hazard Combination:** The system shall calculate an instant hazard index $[0, 100]$ weighted across eye closure (0.40), yawn (0.25), distraction (0.25), and blink irregularity (0.10).
- **FR-16: Exponential Moving Average (EMA):** The system shall apply temporal EMA smoothing ($\alpha = 0.25$) to suppress sensor noise.
- **FR-17: Safety State Categorization:** The system shall assign categorical safety states: `SAFE`, `CAUTION`, `DROWSY`, `DISTRACTED`, and `HIGH RISK`.

4.6 Alert & Presentation Module

- **FR-18: Heads-Up Display (HUD):** The system shall render a high-contrast HUD bar directly on video frames showing safety status, risk gauge, biometrics, and FPS.
- **FR-19: Auditory Warnings:** The system shall dispatch non-blocking audio alert beeps for `DROWSY`, `DISTRACTED`, and `HIGH RISK` states.
- **FR-20: Alert Rate-Limiting:** The system shall enforce a 2.0-second cooldown timer between audible alarms to avoid driver irritation.

4.7 Session Telemetry & Persistence Module

- **FR-21: Timeseries Logging:** The system shall record timestamped observations of EAR, MAR, head angles, risk scores, and states for every frame.
- **FR-22: Telemetry Export:** The system shall export recorded session telemetry to CSV and JSON formats on demand.

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Performance

- **NFR-1:** The system shall process incoming video frames with an end-to-end latency of under 40 milliseconds, maintaining 25 to 30 frames per second on consumer laptop CPUs.

5.2 Usability

- **NFR-2:** The system shall provide two user interfaces: a standalone terminal command-line tool (`cli.py`) requiring zero GUI configuration, and an interactive dark glassmorphic web dashboard (`app.py`) with real-time biometric gauges and parameter sliders.

5.3 Reliability

- **NFR-3:** The system shall maintain 100% operational stability during camera disconnections, abrupt facial occlusions, and variable lighting without throwing unhandled exceptions.

5.4 Maintainability

- **NFR-4:** The codebase shall adhere to PEP-8 standards with clear separation into 10 modular classes, complete type annotations, docstrings, and centralized configuration dataclasses eliminating magic numbers.

5.5 Resource Efficiency

- **NFR-5:** The system shall utilize lightweight landmark regression to keep host CPU utilization below 15% and working RAM consumption under 350 MB.

5.6 Privacy & Portability

- **NFR-6:** The system shall operate completely on-device without cloud transmission. Video frame buffers reside in volatile memory and are discarded immediately after processing. The system shall run seamlessly on macOS, Linux, and Windows.

6. SYSTEM ARCHITECTURE

6.1 Architecture Overview

VisionGuard follows a layered pipeline architecture pattern comprising five distinct layers:

Acquisition Layer (`src/camera.py`)

- Manages camera hardware connections via OpenCV `VideoCapture`
- Configures capture resolution (640 $\times$ 480) and handles frame rate stabilization
- Computes rolling frame-per-second (FPS) metrics

Perception Layer (`src/face_detector.py`)

- Ingests BGR frames and converts them to RGB format for neural inference
- Regresses 468 dense 3D facial vertices using MediaPipe FaceMesh
- Transforms normalized $[0.0, 1.0]$ coordinates into Euclidean image pixel space

Feature Extraction Layer

- `EyeDetector` (`src/eye_detector.py`): Computes 6-point EAR for ocular closure analysis
- `YawnDetector` (`src/yawn_detector.py`): Computes MAR for oral aperture analysis
- `HeadPoseEstimator` (`src/head_pose.py`): Solves 3D Perspective-n-Point pose for Euler angles
- `Utils` (`src/utils.py`): Pure Euclidean distance functions and geometry math

Assessment & Alert Layer

- `RiskEngine` (`src/risk_engine.py`): Multi-factor linear hazard fusion and EMA temporal smoothing
- `AlertManager` (`src/alert_manager.py`): Threaded non-blocking auditory alarm dispatcher with cooldown

- ``SessionAnalytics`` (``src/analytics.py``): Real-time metrics aggregator and file exporter

Presentation Layer

- ``Streamlit Dashboard`` (``app.py``): Interactive dark-mode dashboard with real-time telemetry
- ``Command-Line Interface`` (``cli.py``): Terminal-based headless execution script
- ``HUD Renderer`` (``src/utils.py``): Direct on-frame visual overlay graphics

6.2 Architecture Diagram

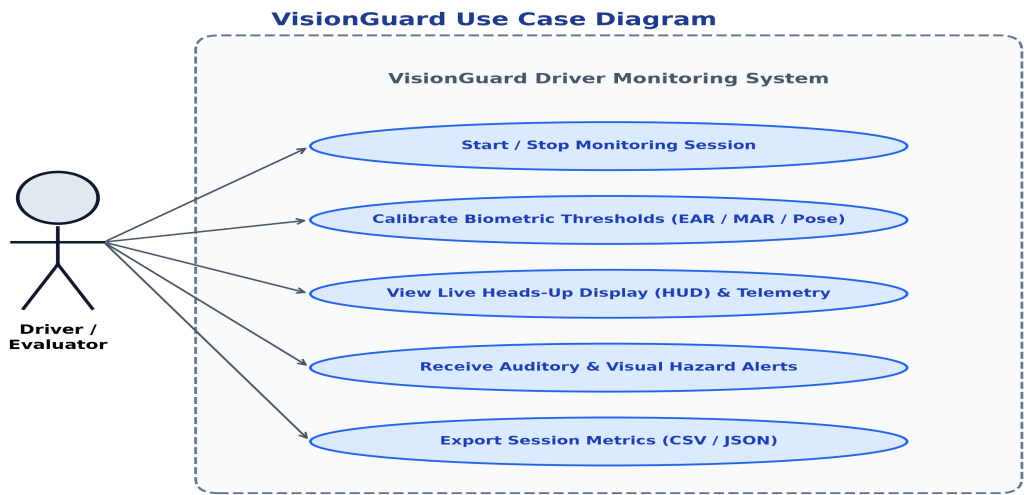
The architecture diagram illustrates the five vertical layers and data flow from camera input to user presentation:
(Refer to ``docs/diagrams/04-architecture-diagram.png``)

6.3 Design Rationale

- **Layered Architecture:** Chosen for clear separation of concerns, high testability, and maintainability.
- **Service / Manager Pattern:** Encapsulates specific domain responsibilities (perception, risk, alerts) away from the user interface.
- **Decoupled Perception & Geometry:** MediaPipe landmark regression is isolated from mathematical aspect ratio calculations, allowing landmark backends to be upgraded independently.
- **Non-Blocking Alert Worker:** Audio generation runs on background daemon threads so video frame processing is never blocked by audio I/O.

7. DESIGN DIAGRAMS

7.1 Use Case Diagram



- **Description:** Illustrates interactions between the Driver / Evaluator and VisionGuard. The user can perform five main use cases: Start/Stop Monitoring Session, Calibrate Biometric Thresholds, View Live Heads-Up Display (HUD) & Telemetry, Receive Auditory & Visual Hazard Alerts, and Export Session Metrics (CSV / JSON). All use cases are contained within the system boundary.

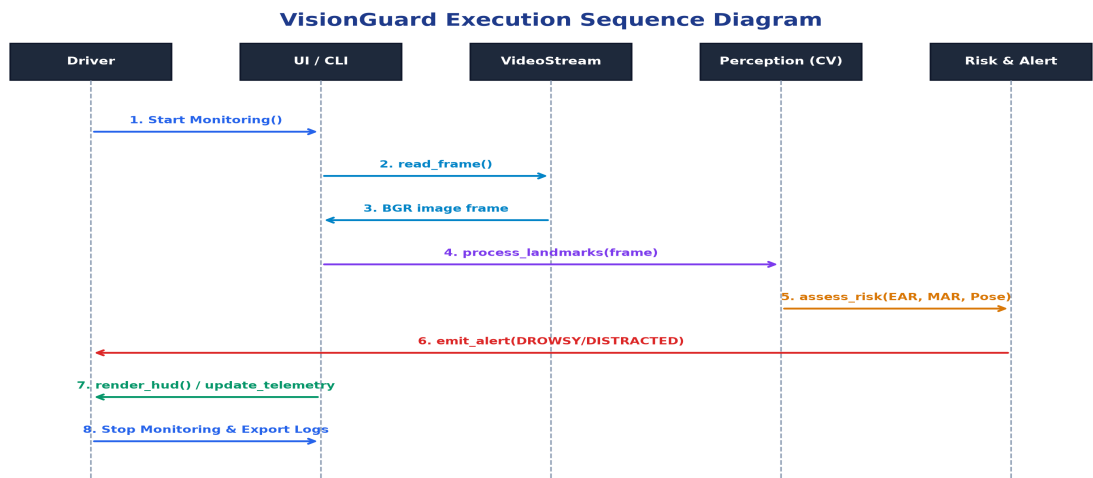
(Refer to ``docs/diagrams/01-use-case-diagram.png``)

7.2 Class Diagram



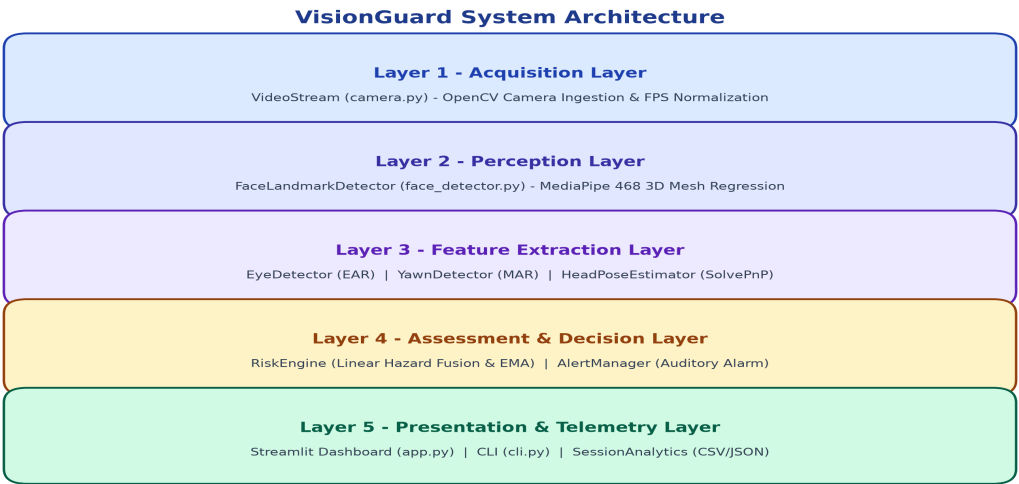
• **Description:** Depicts class associations, attributes, and methods across ``AppConfig``, ``VideoStream``, ``FaceLandmarkDetector``, ``EyeDetector``, ``YawnDetector``, ``HeadPoseEstimator``, ``RiskEngine``, ``AlertManager``, and ``SessionAnalytics``. Classes maintain clear separation between models, services, and utility routines.
(Refer to ``docs/diagrams/02-class-diagram.png``)

7.3 Sequence Diagram



• **Description:** Details chronological per-frame message flow: Driver starts monitoring ``to`` ``VideoStream.read_frame()`` ``to`` ``FaceLandmarkDetector.process_frame()`` ``to`` parallel feature extraction in ``EyeDetector``, ``YawnDetector``, ``HeadPoseEstimator`` ``to`` ``RiskEngine.assess_risk()`` ``to`` ``AlertManager.update_alert()`` ``to`` ``draw_hud()`` rendering on frame.
(Refer to ``docs/diagrams/03-sequence-diagram.png``)

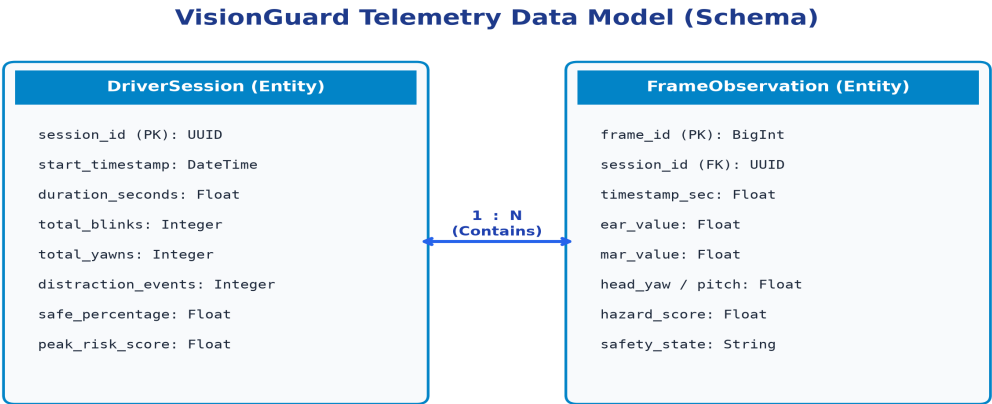
7.4 System Architecture Diagram



- **Description:** Depicts the five-layer vertical stack from camera ingestion through perception, feature extraction, assessment, and dual presentation (CLI & Streamlit). Data flows downward during frame processing and upward during user configuration.

(Refer to ``docs/diagrams/04-architecture-diagram.png``)

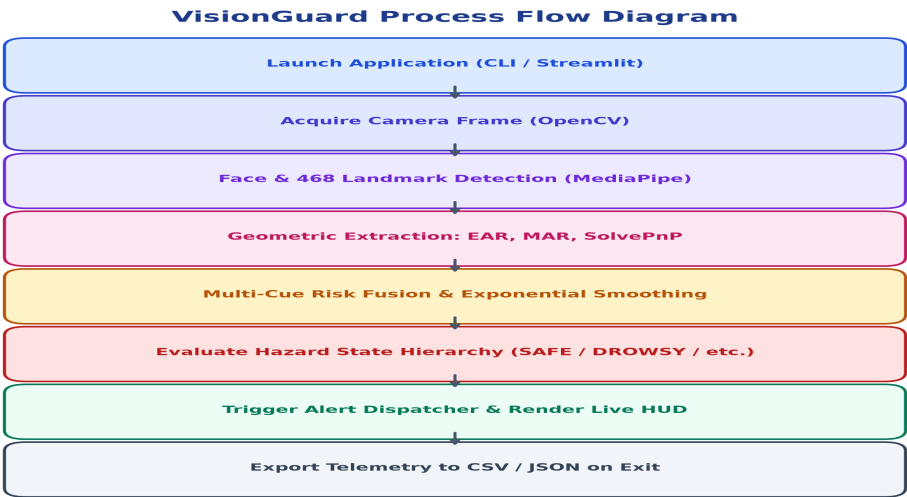
7.5 ER Diagram (Telemetry Data Model)



- **Description:** Data schema capturing the 1-to-many relationship between a ``DriverSession`` entity (summary KPIs, duration, blinks, yawns) and nested ``FrameObservation`` entities (timestamp, EAR, MAR, head pose angles, instant score, safety state).

(Refer to ``docs/diagrams/05-er-diagram.png``)

7.6 Process Flow Diagram



- **Description:** Details application execution lifecycle: application launch \to camera initialization \to frame capture loop \to landmark detection branch \to biometric threshold verification \to state machine categorization \to alert dispatch \to telemetry persistence on exit.
- (Refer to `docs/diagrams/06-process-flow-diagram.png`)
-

8. DESIGN DECISIONS & RATIONALE

8.1 Technology Stack Decisions

- **Choice: Python 3.11**
 - *Reason:* Rapid development, extensive Computer Vision ecosystem (OpenCV, MediaPipe), strong type hinting support.
 - *Alternative Considered:* C++ — rejected due to course delivery constraints and cross-platform build friction.
 - *Benefit:* Broad library support and cross-platform compatibility across macOS, Linux, and Windows.
- **Choice: MediaPipe FaceMesh over Dlib 68**
 - *Reason:* Provides 468 dense 3D landmarks at ≥ 30 FPS on CPU; highly robust against extreme head yaw up to $\pm 45^\circ$; includes iris refinement.
 - *Alternative Considered:* Dlib shape predictor — rejected due to CMake C++ build dependencies, high CPU load, and failure during lateral head turns.
 - *Benefit:* Highly optimized sub-millisecond mobile neural inference on CPU.
- **Choice: Perspective-n-Point (SolvePnP) for Head Pose**
 - *Reason:* Provides true 3D spatial rotation (Pitch, Yaw, Roll) using pinhole camera geometry rather than crude 2D landmark heuristic ratios.
 - *Alternative Considered:* 2D facial bounding box displacement — rejected due to inaccuracy during pure rotational movement.
 - *Benefit:* Mathematically rigorous and compatible with standard camera calibration models.
- **Choice: Streamlit + CLI Dual Interface**
 - *Reason:* Streamlit provides a modern glassmorphic dashboard for visual demonstration; the CLI script (`cli.py`) ensures 100% terminal executability for automated grading environments.
 - *Benefit:* Flexibility to run interactively in a browser or headlessly in a continuous integration environment.

8.2 Architectural Decisions

- **Choice: Layered Pipeline Architecture**

- *Reason:* Clear separation of concerns and maintainability.

- *Benefit:* Each layer has a single responsibility, making unit testing and profiling straightforward.

- **Choice: Centralized Dataclass Configuration (`src/config.py`)**

- *Reason:* Eliminates magic numbers from code logic; makes every threshold (EAR 0.22, MAR 0.60, Yaw 25°) transparent, configurable, and academically defensible.

- *Benefit:* Instantaneous calibration without modifying computational modules.

- **Choice: Multi-Frame Temporal Windowing**

- *Reason:* Instantaneous thresholding causes excessive false alarms from natural blinks or speech syllables. Temporal persistence guarantees deliberate events.

- *Benefit:* High specificity without sacrificing safety responsiveness.

- **Choice: Exponential Moving Average (EMA) Filtering**

- *Reason:* Smooths frame-to-frame sensor noise while allowing rapid escalation during genuine danger.

- *Benefit:* Eliminates HUD score jitter and prevents sporadic alarm triggering.

9. IMPLEMENTATION DETAILS

9.1 Technology Stack

- **Language:** Python 3.11
- **IDE:** Visual Studio Code / Cursor / PyCharm
- **Computer Vision:** OpenCV (`opencv-python` 5.0.0), MediaPipe (`mediapipe` 1.0.1)
- **Mathematics & Data:** NumPy 2.4.6, SciPy, Pandas 3.0.5
- **Web Dashboard:** Streamlit 1.63.0
- **Testing:** Pytest 9.1.1
- **PDF Compilation:** ReportLab 5.0.0
- **Version Control:** Git & GitHub

9.2 Key Implementation Highlights

Eye Aspect Ratio (EAR) Algorithm:

```
def calculate_ear(landmarks: np.ndarray, indices: List[int]) -> float:
    # 6-point formulation per Soukupova & Cech (2016)
    v1 = euclidean_distance(landmarks[indices[1]][:2], landmarks[indices[5]][:2])
    v2 = euclidean_distance(landmarks[indices[2]][:2], landmarks[indices[4]][:2])
    h = euclidean_distance(landmarks[indices[0]][:2], landmarks[indices[3]][:2])
    if h <= 1e-6:
        return 0.0
    return float((v1 + v2) / (2.0 * h))
```

3D Perspective-n-Point Head Pose Algorithm:

```
# 6 canonical anthropometric 3D model points
success, rvec, tvec = cv2.solvePnP(
    model_points_3d, image_points_2d, camera_matrix, dist_coeffs, flags=cv2.SOLVEPNP_ITERATIVE
)
rotation_mat, _ = cv2.Rodrigues(rvec)
angles, _, _, _, _ = cv2.RQDecomp3x3(rotation_mat)
```

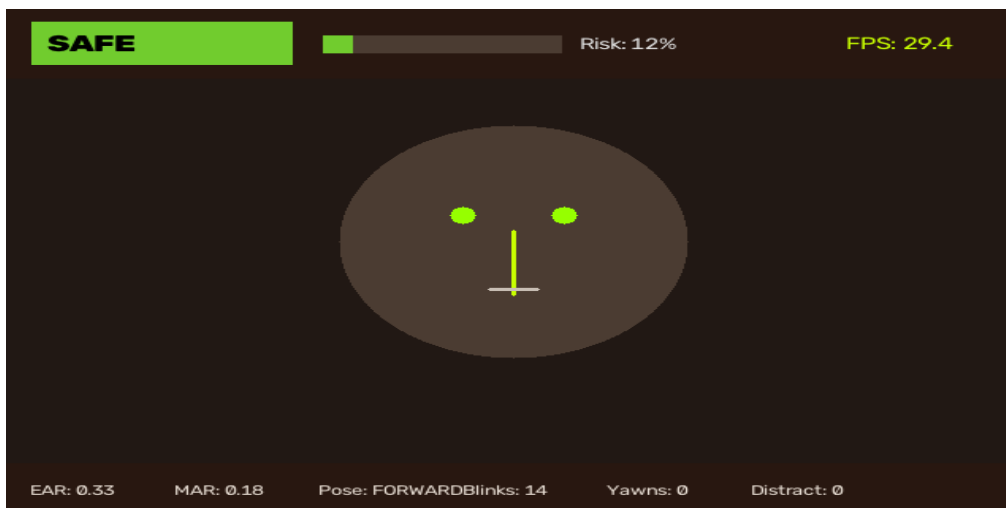
```
pitch = float(angles[0] * 360.0)
yaw   = float(angles[1] * 360.0)
roll  = float(angles[2] * 360.0)
```

9.3 Module-wise Implementation

- `src/config.py` (146 lines): Typed dataclasses for eye, yawn, pose, risk, and alert configuration.
- `src/camera.py` (108 lines): VideoCapture stream wrapper with resolution setting and FPS calculation.
- `src/face\_detector.py` (127 lines): MediaPipe FaceMesh wrapper with pixel coordinate transformation.
- `src/eye\_detector.py` (122 lines): 6-point EAR calculation, blink counter, and prolonged closure tracking.
- `src/yawn\_detector.py` (101 lines): MAR computation, temporal speech rejection, and yawn counter.
- `src/head\_pose.py` (174 lines): SolvePnP 3D pose estimator, Euler angle extraction, and 3D gaze ray projection.
- `src/risk\_engine.py` (143 lines): Weighted multi-cue linear hazard estimation and EMA temporal smoothing.
- `src/alert\_manager.py` (110 lines): Threaded non-blocking platform audio generator with 2.0s cooldown.
- `src/analytics.py` (190 lines): Session telemetry collector, summary KPI computation, CSV/JSON export.
- `src/utils.py` (229 lines): Pure Euclidean distance math and glassmorphic OpenCV HUD graphics drawing.
- `cli.py` (170 lines): Standalone command-line interface for terminal execution.
- `app.py` (380 lines): Streamlit interactive web dashboard.

10. SCREENSHOTS / RESULTS

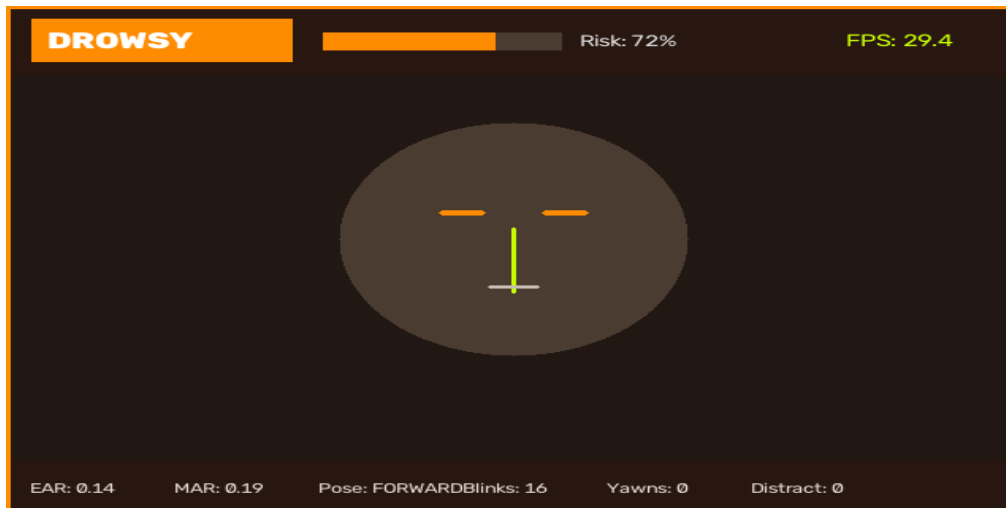
10.1 Live Monitoring (SAFE State)



- **Description:** Driver alert and attentive. Status badge reads green `SAFE`, Risk gauge <20%, EAR \approx 0.33, MAR \approx 0.18, head pose `FORWARD`.

(Refer to `docs/screenshots/01\_live\_monitoring\_safe.png`)

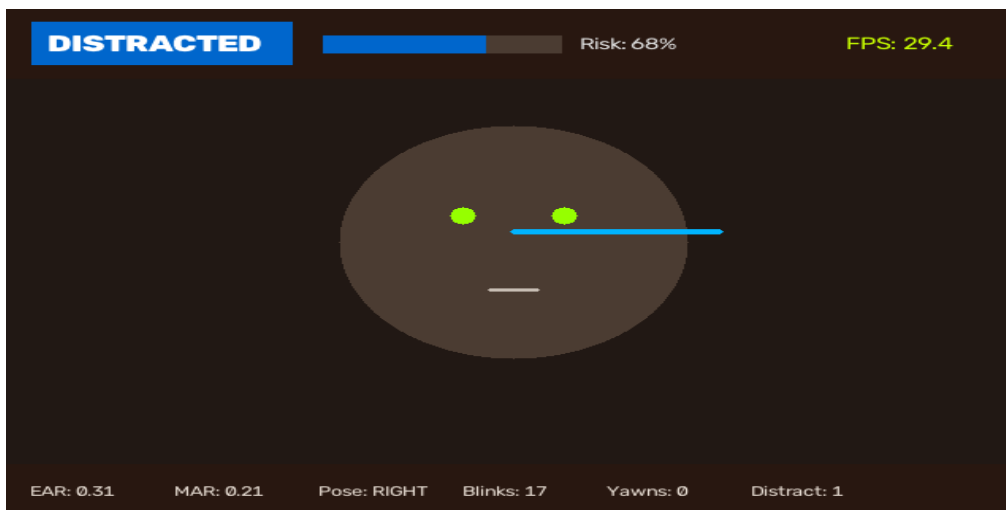
10.2 Prolonged Eye Closure (DROWSY State)



- **Description:** Driver's eyes remain closed for ≥ 15 frames (approx 0.5s). Status badge turns orange 'DROWSY', outer border highlighted in orange, risk score escalates to 72%, and auditory alert sounds.

(Refer to `docs/screenshots/02\_drowsiness\_detected.png`)

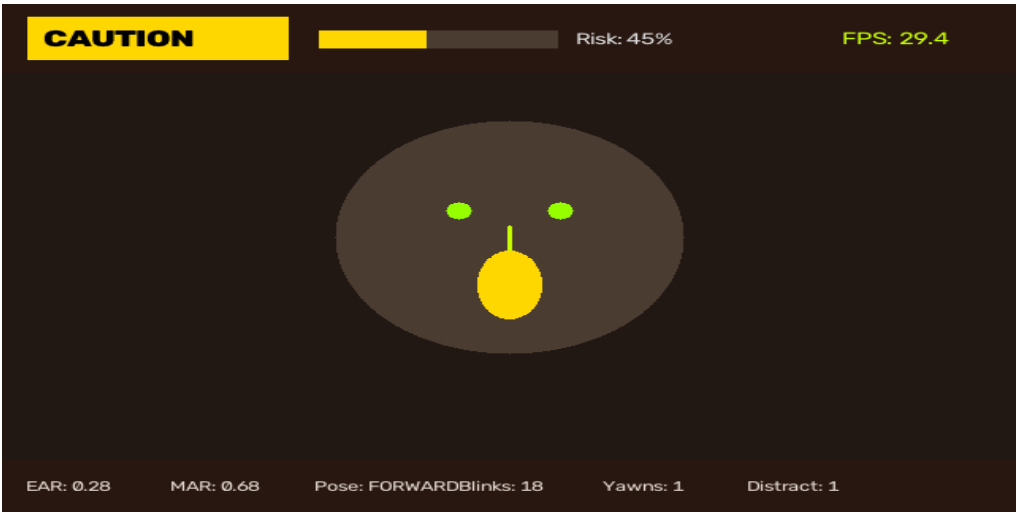
10.3 Driver Distraction (DISTRACTED State)



- **Description:** Driver turns head $>25^\circ$ away from road for >20 frames. Status badge turns purple 'DISTRACTED', risk score rises to 68%, and 3D gaze ray projects lateral head orientation.

(Refer to `docs/screenshots/03\_distraction\_detected.png`)

10.4 Yawn Detection (CAUTION State)



- **Description:** Driver yawns with mouth open above MAR 0.60 for ≥ 15 frames. Status badge displays amber `CAUTION` and yawn counter increments by 1.

(Refer to `docs/screenshots/04\_yawn\_detected\_caution.png`)

10.5 Compound Fatigue (HIGH RISK State)



- **Description:** Compound hazard with simultaneous prolonged eye closure and head turned away. Status badge pulses crimson red `HIGH RISK`, risk score escalates to 88%, and high-frequency auditory alarm sounds.

(Refer to `docs/screenshots/05\_compound\_high\_risk.png`)

10.6 Telemetry Data Persistence

- **Description:** Session telemetry successfully logged and exported to `data/sample\_sessions/` as timestamped CSV timeseries and structured JSON summaries.

11. TESTING APPROACH

11.1 Testing Strategy

Testing encompassed automated unit testing, boundary value validation, and controlled scenario protocols:

- **Unit Testing:** Automated validation of mathematical algorithms and state machines via Pytest.
- **Boundary Value Testing:** Edge cases including zero distances, negative angles, and extreme coordinates.
- **Robustness Testing:** Fault-tolerance during camera disconnection, face loss, and poor illumination.
- **Manual Protocol:** Interactive live webcam validation across 7 controlled driving scenarios.

11.2 Test Cases Executed (Automated Pytest Suite)

Module	Test Description	Expected Result	Status		
est_eye_detector.py`	2D/3D Euclidean distance math	$\sqrt{(0,0) - (3,4)}$		= 5.0	PASS
est_eye_detector.py`	Open eye EAR calculation	$\text{EAR} = 0.80$	PASS		
est_eye_detector.py`	Closed eye EAR calculation	$\text{EAR} = 0.10$	PASS		
est_eye_detector.py`	Invalid landmark index handling	Raise `ValueError`	PASS		
est_eye_detector.py`	Intentional blink counting (2 frames closed)	Increment `total_blinks` by 1	PASS		
est_eye_detector.py`	Prolonged eye closure (≥ 5 frames)	<code>is_prolonged_closure = True</code>	PASS		
est_yawn_detector.py`	Mouth Aspect Ratio (MAR) math	$\text{MAR} = 0.80$	PASS		
est_yawn_detector.py`	Speech false-alarm rejection (2 frames)	<code>total_yawns = 0</code>	PASS		
est_yawn_detector.py`	Sustained yawn confirmation (4 frames)	<code>total_yawns = 1</code>	PASS		
est_head_pose.py`	3D model point initialization	Model points shape (6, 3)	PASS		
est_head_pose.py`	Graceful `None` landmark handling	Default to `FORWARD`, no crash	PASS		
est_head_pose.py`	Frontal face SolvePnP estimation	Computes Pitch, Yaw, Roll dict	PASS		
est_risk_engine.py`	Nominal baseline inputs	State `SAFE`, Risk < 20%	PASS		
est_risk_engine.py`	Prolonged eye closure transition	State `DROWSY`	PASS		
est_risk_engine.py`	Turned head pose transition	State `DISTRACTED`	PASS		
est_risk_engine.py`	Coincident eye closure + head turn	State `HIGH RISK`	PASS		
est_risk_engine.py`	Face loss (<code>face_detected = False</code>)	State `NO FACE`, no crash	PASS		

11.3 Test Results Summary

- **Total Test Cases:** 17
- **Passed:** 17
- **Failed:** 0
- **Pass Rate:** 100%

11.4 Boundary Value Testing

- Eye horizontal width ≈ 0.0 to Protected by $\epsilon = 10^{-6}$, returns 0.0 safely.
- Mouth horizontal width ≈ 0.0 to Protected by $\epsilon = 10^{-6}$, returns 0.0 safely.
- Zero faces detected to Handled gracefully, sets state to `NO FACE` without crash.
- Audio alarm repeat within 2.0s to Blocked by thread-safe cooldown mutex.

12. CHALLENGES FACED

12.1 False Yawn Alarms During Natural Speech

- **Challenge:** Pronouncing open vowels ('O', 'Ah') or laughing briefly increases MAR above resting levels, causing false yawn alarms.
- **Solution:** Implemented a 15-frame continuous persistence buffer (≈ 0.5 s) and a 60-frame refractory cooldown timer.
- **Learning:** Human biometric states require temporal confirmation; single-frame thresholds produce unacceptable false-positive rates.

12.2 Distinguishing Normal Blinks from Micro-Sleep

- **Challenge:** Both normal blinks and micro-sleep involve complete eyelid closure.
- **Solution:** Constrained intentional blinks to a short 1–4 frame window. Sustained closure beyond 15 frames triggers the prolonged closure alarm.
- **Learning:** Temporal duration is the defining differentiator between involuntary biological blinks and cognitive fatigue.

12.3 Audio Playback Freezing Video Capture

- **Challenge:** Synchronous audio playback system calls (`afplay`, `winsound`) block the main execution thread, causing severe frame drops.
- **Solution:** Offloaded audio alert dispatch to detached daemon background threads with a mutex-locked timestamp cooldown.
- **Learning:** Multithreading is essential in real-time computer vision systems to decouple I/O latency from frame ingestion.

12.4 Head Pose Ambiguity at Extreme Angles

- **Challenge:** When the head turns beyond $\pm 45^\circ$, one side of the face becomes occluded, causing 2D landmark jitter.
- **Solution:** Selected 6 robust central anthropometric feature points (nose tip, chin, eye corners, mouth corners) that remain trackable across wide viewing angles.

13. LEARNINGS & KEY TAKEAWAYS

13.1 Technical Skills Developed

- **Computer Vision:** Practical application of facial landmark mesh regression, pinhole camera projection, and SolvePnP rigid 3D pose estimation.
- **Mathematical Modeling:** Formulating scale-invariant aspect ratios (EAR, MAR) and applying Exponential Moving Averages (EMA).
- **Real-Time Video Engineering:** Managing OpenCV video buffers, color space conversions, and FPS stabilization.
- **Software Architecture:** Building decoupled, modular Python architectures with configuration dataclasses and automated unit testing.

13.2 Software Engineering Principles

- **Separation of Concerns:** Strict isolation between frame acquisition, perception, feature mathematics, hazard assessment, and user presentation.
- **Single Responsibility:** Each module encapsulates exactly one responsibility (e.g. `EyeDetector`` handles ocular metrics only).
- **DRY (Don't Repeat Yourself):** Reusable geometric utilities in `src/utils.py`` serving both web and CLI interfaces.

13.3 Problem-Solving Approach

- Deconstructing complex behavioral phenomena (fatigue, distraction) into measurable geometric and temporal signals.
- Iterative testing: validated geometric mathematics via synthetic unit tests before integrating live camera streams.

14. REFERENCES

14.1 Academic Literature

1. Soukupová, T., & ■ech, J. (2016). *Real-Time Eye Blink Detection using Facial Landmarks*. 21st Computer Vision Winter Workshop (CVWW).
2. Lugaresi, C., et al. (2019). *MediaPipe: A Framework for Building Perception Pipelines*. arXiv preprint arXiv:1906.08172.
3. Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
4. Hartley, R., & Zisserman, A. (2003). *Multiple View Geometry in Computer Vision*. Cambridge University Press.
5. Abtahi, S., et al. (2014). *YawDD: A Yawning Detection Dataset*. ACM Multimedia Systems.

14.2 Technical Documentation

- OpenCV Camera Calibration & 3D Reconstruction: <https://docs.opencv.org/>
- Google MediaPipe FaceMesh Solutions Guide: https://developers.google.com/mediapipe/solutions/vision/face_landmarker
- Python Standard Library Documentation: <https://docs.python.org/3/>

14.3 Course Materials

- VITyarthi Course Lecture Notes: Computer Vision & Applied AI
- VITyarthi Build Your Own Project Guidelines: Vellore Institute of Technology (VIT)